

DATA PREPROCESSING PIPELINE

Detailed User Manual

© Andrei Tregubov, 2026.

1. Introduction	4
1.1 Purpose of the Software	4
1.2 Input Data.....	4
1.2.1 Data File Format	4
1.2.2 The .INFO Metadata File	5
1.2.3 Loading Files into the Program.....	5
1.3 Overall Data Processing Pipeline	6
1.4 Dive Detection and Removal	6
1.4.1 The Problem	6
1.4.2 Detection Algorithm.....	6
1.4.3 Guard Conditions	7
1.4.4 Visualisation and Manual Correction	7
1.5 Subsampling and the "Build Full Data" Button	7
1.5.1 Subsampled Cache for Interactive Work	7
1.5.2 The "Build Full Data" Button	8
1.6 Fourier Transform: Complete Description	9
1.6.1 Why Fourier Filtering is Needed	9
1.6.2 Forward Fourier Transform (Step 3, Spectrum Computation).....	9
1.6.3 Spectrum Visualisation.....	9
1.6.4 Cutoff Frequency Selection	9
1.6.5 Mirror Padding and Inverse Transform	9
1.6.6 Before / After Comparison Window	10
1.7 Array Pass and Wave Parameter Computation.....	11
1.7.1 Pass Structure	11
1.7.2 RMS Filtering	11
1.7.3 Spike Removal.....	11
1.7.4 Spline Interpolation	11
1.7.5 Standard Deviation, As and Hs	12
1.7.6 Individual Amplitudes, Wave Heights, Tz	12
1.7.7 As _{1/3} and Hs _{1/3} — Significant Wave Parameters from Individual Statistics.....	12
1.7.8 Dispersion Equation Solution (kh, k).....	13
1.7.9 Geometric and Nonlinear Parameters	13
1.7.10 Spectral Parameters (Q, v, ϵ_w).....	14
1.7.11 Regularity Parameter ρ	14
1.7.12 Benjamin–Feir Index (BFI)	15
1.8 Program Output.....	16
2. Step-by-Step User Guide	17
2.1 Preparation.....	17
2.1.1 System Requirements.....	17

2.1.2 Launching the Program	17
2.1.3 Output Folder and Checkpoint System	17
2.2 Step 1 — Loading and Converting Files	18
2.2.1 Adding Files	18
2.2.2 Starting Conversion.....	18
2.2.3 Completion Message	19
2.3 Step 2 — Visualisation and Trimming	20
2.3.1 Reading the Graph.....	20
2.3.2 Manual Trimming	20
2.3.3 What Happens to the Data	21
2.4 Step 3 — Fourier Filtering.....	22
2.4.1 Spectrum Computation.....	22
2.4.2 Cutoff Frequency Selection	22
2.4.3 Applying the Filter and Comparison Window	23
2.5 Step 4 — Analysis and Parameter Computation	24
2.5.1 Settings Before Starting	24
2.5.2 Real-Time Processing	24
2.5.3 Pipeline Complete Window	25
2.6 Exporting Results	26
2.6.1 Loading Results in MATLAB	26
2.6.2 Loading Results in Python.....	27

1. Introduction

1.1 Purpose of the Software

Data Preprocessing Pipeline is a desktop GUI application designed for the **complete processing cycle of bottom-mounted pressure sensor data**. It accepts raw pressure recordings in metres of water column, sequentially cleans them of artefacts, applies Fourier filtering, and computes a full set of wave parameters for each 20-minute record.

The primary application area is physical oceanography and marine hydrology: **wind wave analysis** from pressure records of moored buoys or seafloor platforms. A pressure sensor positioned at the bottom or within the water column registers hydrostatic pressure variations caused by passing surface waves. After conversion to surface displacement, these data allow characterisation of the sea state.

The software fully automates the following tasks, traditionally performed manually in MATLAB or Python:

- Conversion of .dat files into a unified time series with timestamps, split into statistically homogeneous 20-minute records
- Visual inspection and manual trimming of data at the start and end of the recording
- Trend removal and computation of the zero-mean surface displacement signal
- Computation of the full Fourier spectrum and interactive selection of the cutoff frequency
- Inverse Fourier transform with mirror padding to suppress edge artefacts (Gibbs effect)
- Automatic removal of low-energy records and correction of spikes
- Computation of 20+ wave parameters for each record
- Export of individual wave amplitude and height arrays

1.2 Input Data

1.2.1 Data File Format

The program accepts a **set of .dat text files** and one **.INFO metadata file**. Each .dat file contains one number per line — the **length of the water column between the sensor and the water surface, in metres**. This value allows the sensor installation depth to be calculated and the surface displacement to be derived.

⚠ Note: If the raw sensor data are recorded in pressure units (Pa, kPa, dbar, mmHg, etc.), a hydrostatic conversion to metres of water column must be performed **BEFORE** loading into the software: $h = P / (\rho \cdot g)$, where $\rho \approx 1025 \text{ kg/m}^3$ is the density of seawater and $g = 9.81 \text{ m/s}^2$. The program expects values already expressed in metres.

Example .dat file contents:

```
8.294242314722688 ← water column height, m (dot)
8.276029385536757
8,359068152086337 ← comma is also accepted
8.534720557960874
```

The decimal separator may be either a **dot** or a **comma** — the program detects both automatically. This is important for files produced by software with a Russian locale, which uses comma by default. Lines that cannot be parsed as a number are silently skipped, so headers and text comments in the file do not interfere with loading.

Each .dat file corresponds to a continuous block of recording. The sensor produces a **uniformly sampled** signal at a constant sampling rate (e.g. 1 Hz or 8 Hz). There may be multiple .dat files — the program merges them in chronological order of their filenames into a single array.

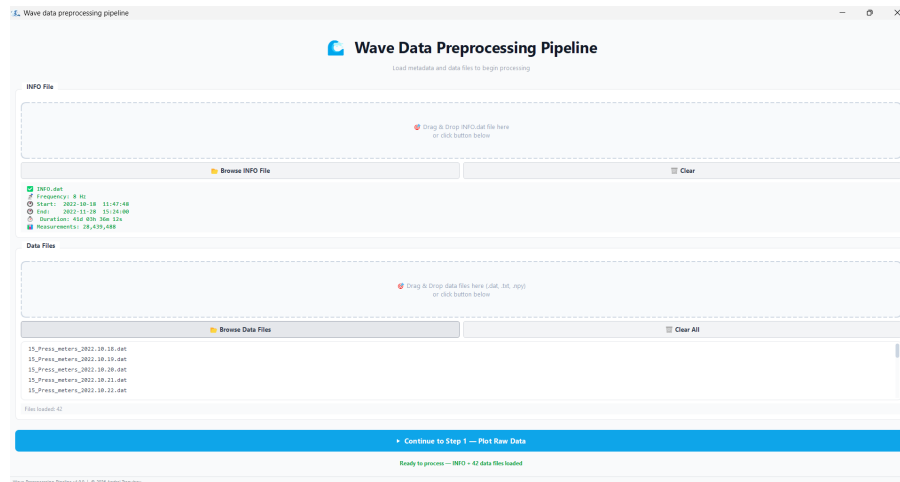


Fig. 1 — Load window with .dat and .INFO files added

1.2.2 The .INFO Metadata File

The .INFO file is a plain-text file of arbitrary structure containing recording metadata. The program reads it using **regular expressions**, so the **order of fields does not matter** — the required data will be found wherever they appear. The file may contain any additional text, comments or fields — everything else is ignored.

The program extracts the following mandatory data from the .INFO file:

- **Sensor sampling frequency.** Expected as a line such as `Частота опроса: 8 Гц` or `Frequency: 8 Hz`. If not found — loading terminates with an error.
- **Recording start date and time.** Expected in the format `YYYY.MM.DD HH:MM:SS`, e.g. `2022.10.18 11:47:48.000`. The first date found is taken as the start.
- **Recording end date and time.** The second date in the same format. Used only for data integrity checking.

The file encoding may be UTF-8, Windows-1251 (cp1251) or Latin-1 — the program tries all variants automatically. The sensor installation depth is **not read from the .INFO file**: it is computed from the data themselves as the mean value of the signal for each 20-minute record.

1.2.3 Loading Files into the Program

To load data:

1. **Launch the program** — the main Step 1 window (Load & Convert) opens.
2. Click **"Add Files"** or drag and drop .dat and .INFO files directly into the program window.
3. Verify that the list contains exactly **one .INFO file** and the required .dat files.

4. Click **"Process & Convert"**. The program merges all .dat files into a single array, generates timestamps starting from the date in .INFO, and splits the series into 20-minute blocks (reading_number).

⚠ **Note:** If the generated timestamp of the last data point differs from the expected recording end time from .INFO by more than 1 hour, the program will display a warning showing both the actual and expected times. This may indicate an incomplete set of .dat files or an error in the .INFO file.

1.3 Overall Data Processing Pipeline

The program is organised as a sequential four-step pipeline. Each step saves its result to the Output folder as a CSV file, allowing processing to be resumed from any step without repeating the previous ones.

Step	Name	Input file	Output file(s)
Step 1	Load & Convert	.dat + .INFO	Step1_TXTtoCSV.csv
Step 2	Visualize & Cut	Step1_TXTtoCSV.csv	Step2_Zero_Mean.csv
Step 3	Fourier Filter	Step2_Zero_Mean.csv	Step3_Transformed.csv
Step 4	Analyze	Step3_Transformed.csv	Parameters.csv Amplitudes_Heights.csv

1.4 Dive Detection and Removal

1.4.1 The Problem

During deployment and retrieval of a bottom-mounted sensor, the beginning and end of a recording inevitably contain **"dive legs"** — transitional sections where the sensor moves through the water column from the surface to the bottom and back. During these moments the pressure signal represents a **monotonically increasing or decreasing trend**, not wave motion. These sections must be identified and excluded before any wave analysis.

1.4.2 Detection Algorithm

The program uses **gradient analysis** to automatically detect dive legs:

Step 1. Gradient and threshold computation:

```
gradient[i] = (y[i+1] - y[i-1]) / 2
```

numerical gradient (central differences)

```
threshold = sensitivity × std(gradient)
```

sensitivity = 3.0 by default

Step 2. Beginning leg (search in the first 40% of data):

The program searches for all moments of a sharp positive gradient spike where simultaneously:

- `gradient[i] > threshold` — significant spike
- `y[i-1] < 2.0 m` — sensor was near the surface before the spike
- `y[i+1] ≥ 2.0 m` — sensor reached depth after the spike

The leg boundary is set at the **last** such spike (the sensor may surface several times before settling at deployment depth). After the last spike the program walks forward until the gradient drops below `grad_std`, then adds a 10% safety margin. The entire section from start to that point is marked as a dive leg and excluded.

Step 3. Ending leg (search in the last 40% of data):

The same logic but searching for sharp negative spikes:

- `gradient[i] < -threshold`
- `y[i-1] ≥ 2.0 m` — sensor was at depth
- `y[i+1] < 2.0 m` — sensor left the water

The boundary is set at the **first** such event. The program walks backward until the gradient calms, adds a 10% safety margin. The entire tail is marked as a retrieval leg.

1.4.3 Guard Conditions

The algorithm does not search for legs in the following cases:

- If `y[0] ≥ 2.0 m` — the sensor started at depth, no beginning leg
- If `y[-1] ≥ 2.0 m` — the recording ends at depth, no ending leg
- If the array has fewer than 100 points — too short for analysis

1.4.4 Visualisation and Manual Correction

After automatic detection the program highlights the dive legs in **red** on the graph. If the detector fired, the **"Manual trim"** button becomes active, allowing the user to manually adjust the boundaries by clicking on the graph to set the start and/or end of the working section.

If no dive legs are detected — the manual correction button remains inactive. This is normal if the recording has already been pre-trimmed or the sensor was fixed at the required depth from the start.

1.5 Subsampling and the "Build Full Data" Button

Full recordings may contain **hundreds of thousands or millions of points** (e.g. 30 days at 8 Hz ≈ 20 million points). Rendering that many points in matplotlib in real time is not feasible — it would freeze the interface for tens of seconds on every zoom operation.

The program addresses this with a two-level approach:

1.5.1 Subsampled Cache for Interactive Work

When processing each step, the program automatically creates a **subsampled visualisation cache** — a thinned version of the data containing no more than 5 000–10 000 points. Thinning is uniform: every k-th point is retained:

```
step = max(1, len(data) // target_points)
```

target_points = 5 000–10 000 depending on the step

This cache is used for all interactive operations — zooming, panning, selecting trim points. The visual difference between the subsample and the full data at overview scale is negligible.

1.5.2 The "Build Full Data" Button

On each step the toolbar contains a button **"Build all data points (slow)"**. Clicking it loads the full data from CSV and rebuilds the graph without thinning. This is a **slow operation** (may take several seconds), but it allows seeing the exact signal shape — particularly important when trimming the start/end (Step 2) and selecting the cutoff frequency (Step 3).

⚠ Note: When using the trim buttons (setting start and end of the recording), click "Build all data points" first — then your click on the graph will correspond to the true position in the data, not the subsampled version.

1.6 Fourier Transform: Complete Description

1.6.1 Why Fourier Filtering is Needed

After computing the zero-mean surface displacement signal $\eta(t)$, it still contains **low-frequency components** — tidal variations, infragravity waves, slow drifts. These components have periods of hundreds of seconds or more and are unrelated to wind waves. They must be removed for correct wave parameter estimation.

The program applies a **high-pass filter in the frequency domain**: it zeroes out all spectral components below the user-selected cutoff frequency ω_c . Physically this is equivalent to removing all waves with period longer than $T_c = 2\pi / \omega_c$.

1.6.2 Forward Fourier Transform (Step 3, Spectrum Computation)

The program applies the **Fast Fourier Transform (FFT)** to the zero-mean signal — an algorithm that decomposes the time series into sinusoids of different frequencies. Each frequency is assigned an amplitude and a phase. The result is a **complex spectrum**: an array of complex numbers, one per frequency from zero to the Nyquist frequency (half the sampling rate).

Frequencies are expressed in **angular units (rad/s)** rather than Hz — standard convention in wave physics ($\omega = 2\pi \cdot f$). The full spectrum, including real and imaginary parts, is saved to `Step3_Spectrum.csv` — this is required for accurate signal reconstruction during the inverse transform.

⚠ Note: The recommended cutoff: remove components with periods longer than 10 minutes ($\omega < 0.0105$ rad/s). There is no need to touch anything further to the right — wind waves are well above this frequency.

1.6.3 Spectrum Visualisation

The spectrum is displayed in two simultaneous panels:

- **Upper panel** — full spectrum for $\omega > 0.1$ rad/s (linear scale). Wind wave peaks are visible here.
- **Lower panel** — range 0–0.1 rad/s on a logarithmic vertical scale. Tidal and infragravity wave components are visible here.

Because the full spectrum contains as many points as the original signal (potentially millions), **spectrum subsampling** is applied for display, identical to the time series subsampling. The **"Build full spectrum (slow)"** button renders the exact unsubsampled spectrum.

1.6.4 Cutoff Frequency Selection

The cutoff frequency ω_c is selected by **double-clicking on the lower panel**. After the click, the program:

- Draws a vertical red line at the selected frequency on both panels
- Displays the corresponding period: **$T_c = 2\pi / \omega_c$** (in seconds and minutes)
- Shades the area to the left of the line — the components that will be removed
- Activates the **"Apply & Continue"** button

1.6.5 Mirror Padding and Inverse Transform

Directly zeroing low frequencies in the spectrum is equivalent to ideal rectangular filtering. A rectangular window in the frequency domain produces the **Gibbs effect** in the time domain —

oscillations and "ringing" near the signal edges. To suppress this artefact the program uses **mirror padding**.

The idea:

Before filtering, **mirror "tails"** are appended to both ends of the signal. The left tail is the beginning of the signal reversed. The right tail is the end of the signal reversed. As a result the signal becomes longer and its edges are **smoothly continued** without discontinuities.

The length of each tail is chosen equal to one full period of the cutoff frequency — exactly the extent over which the edge artefact decays. The maximum tail length is capped at one quarter of the signal length to avoid inflating the array too much.

Processing steps:

1. Mirror tails are appended to both ends of the signal.
2. FFT is applied to the extended signal.
3. All spectral components below the cutoff frequency are zeroed.
4. Inverse FFT is applied — producing the filtered extended signal.
5. The tails are discarded. They have already absorbed the edge oscillations. The filtered signal of the original length is returned — free of edge artefacts.

1.6.6 Before / After Comparison Window

After the inverse transform the program opens a **"Before / After"** comparison window. Both signals are overlaid on one graph: **orange** — the original signal before filtering, **blue** — the filtered signal. This allows verifying that the filtering is correct: the long-period trend has been removed and the wave shape is preserved.

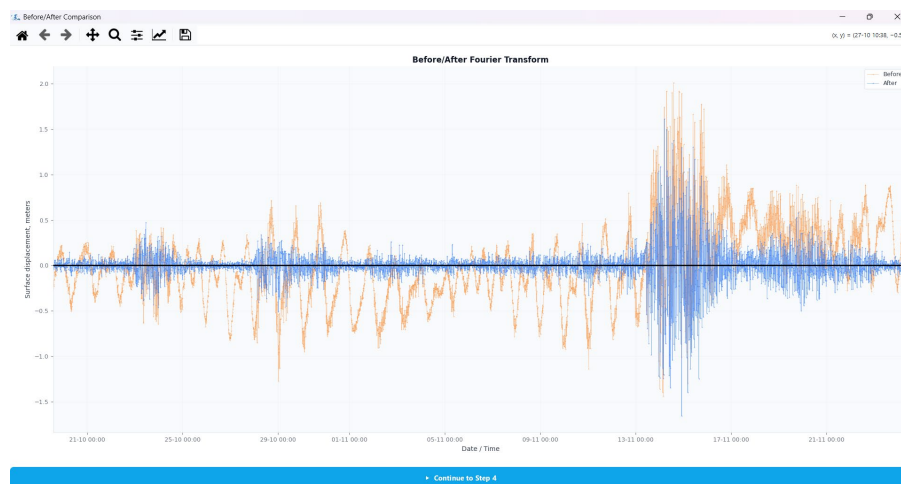


Fig. 2 — Before/After window: orange original and blue filtered signal

1.7 Array Pass and Wave Parameter Computation

1.7.1 Pass Structure

Step 4 operates on the filtered surface displacement signal. The data are divided into **20-minute blocks (readings)** with a unique `reading_number`. The program passes sequentially through each block and computes the full parameter set. For each record the following stages are executed:

- **Stage 1:** RMS filtering (optional)
- **Stage 2:** Spike removal (optional)
- **Stage 3:** Array preparation (optional spline interpolation)
- **Stage 4:** Amplitude parameters (σ , A_s , H_s , $A_{s_1/3}$, $H_{s_1/3}$)
- **Stage 5:** Zero-crossings, individual amplitudes, heights and T_z
- **Stage 6:** Dispersion equation solution (kh , k)
- **Stage 7:** Geometric and nonlinear parameters (ϵ , a , U_r)
- **Stage 8:** Spectral parameters (Q , v , ϵ_w)
- **Stage 9:** Regularity parameter ρ (ρ)
- **Stage 10:** Derived parameters (ϵ_ρ , γ , BFI)

1.7.2 RMS Filtering

If the **"Remove recordings with RMS <"** checkbox is enabled, the root mean square of the signal is computed for each record:

$$\text{RMS} = \sqrt{\text{mean}(\eta^2)}$$

root mean square of surface displacement

If $\text{RMS} < \text{threshold}$ (default 0.015 m), the record is deemed "quiet" — wave energy is too low for reliable analysis. The record is **excluded from further computation** and marked in red on the graph.

1.7.3 Spike Removal

If the **"Remove spikes"** checkbox is enabled, the program detects sudden amplitude jumps characteristic of electronic sensor interference. The threshold is defined via variance:

$$\text{threshold_spike} = 6 \cdot \sqrt{\text{Var}(\eta)}$$

6 standard deviations — robust outlier criterion for marine data

For each adjacent pair of points, if $|\eta[j+1] - \eta[j]| > \text{threshold_spike}$, the point $\eta[j+1]$ is a spike. It is replaced by linear interpolation between its neighbours. Each detected spike is **marked with a circle** on the graph.

1.7.4 Spline Interpolation

If the **"Spline interpolation"** checkbox is enabled and $\text{target_hz} > \text{sensor_freq}$, the program builds a cubic spline on the original signal and resamples it to the higher frequency:

```
arr_amp = CubicSpline(t_orig, arr)(t_new)
```

where $t_{\text{new}} = 0, 1/\text{target_hz}, 2/\text{target_hz}, \dots$

The result `arr_amp` is used for amplitude parameter computation. The interpolation improves accuracy of the maximum estimate within each half-wave without adding new spectral content. All spectral parameters (Q , v , ε) are computed on the **original** array.

1.7.5 Standard Deviation, A_s and H_s

Standard deviation is computed on the interpolated array `arr_amp`:

$$\sigma = \sqrt{\text{Var}(\text{arr_amp})}$$

Significant wave amplitude:

$$A_s = 2\sigma$$

mean of the highest third of amplitudes under narrow-band Gaussian assumption

Significant wave height:

$$H_s = 4\sigma$$

corresponds to the definition $H_{1/3} = 4 \cdot \text{RMS of surface elevation}$

1.7.6 Individual Amplitudes, Wave Heights, T_z

The function `extract_amplitudes_and_heights` finds all zero-crossings of the signal via **PyAstronomy.pyaC.zerocross1d** with sub-sample accuracy (linear interpolation between adjacent samples). Each segment between two consecutive crossings is one **individual half-wave**.

Amplitude of each half-wave:

$$A_i = \max(|\text{arr_amp}[\text{xc}[i] : \text{xc}[i+1]]|)$$

maximum absolute displacement within the segment

Height of each wave (sum of two adjacent amplitudes — crest + trough):

$$H_i = A_i + A_{\{i+1\}}$$

the i -th wave is formed by the i -th and $(i+1)$ -th half-waves

Mean zero-crossing period T_z :

$$T_z = (N / \text{fs}) / N_{\text{crossings}} \times 2$$

$N_{\text{crossings}}$ — number of zero-crossings; factor 2 converts half-periods to periods

1.7.7 $A_{s\ 1/3}$ and $H_{s\ 1/3}$ — Significant Wave Parameters from Individual Statistics

Standard oceanographic significant amplitude and height computed directly from individual wave statistics rather than through σ :

$$As_{1/3} = \text{mean}(\text{sorted}(A_i, \text{desc})[: N/3]) \\ \text{arithmetic mean of the highest third of amplitudes}$$

$$Hs_{1/3} = \text{mean}(\text{sorted}(H_i, \text{desc})[: N/3]) \\ \text{arithmetic mean of the highest third of wave heights}$$

If fewer than 3 waves are detected — fallback: $As_{1/3} = As$, $Hs_{1/3} = Hs$.

1.7.8 Dispersion Equation Solution (kh, k)

For shallow and intermediate water depths the relationship between wave period and wavenumber is described by the **linear wave theory dispersion relation**:

$$\omega^2 = g \cdot k \cdot \tanh(kh) \\ \omega = 2\pi/Tz, g = 9.81 \text{ m/s}^2, h \text{ — depth, } k \text{ — wavenumber}$$

Introducing the dimensionless parameter kh, the equation becomes:

$$kh \cdot \tanh(kh) = 4\pi^2 h / (g \cdot Tz^2) \\ \text{nonlinear equation in kh, solved numerically}$$

The program solves this using **scipy.optimize.fsolve** (Newton–Raphson). The initial guess is chosen adaptively:

- $x_0 = \text{target}$ if target < 1 (shallow water)
- $x_0 = \sqrt[3]{\text{target}}$ if $1 \leq \text{target} < 10$ (intermediate depth)
- $x_0 = \text{target}$ if target ≥ 10 (deep water)

After finding kh, the wavenumber is:

$$k = kh / h$$

1.7.9 Geometric and Nonlinear Parameters

Wave steepness ε :

$$\varepsilon = (k \cdot Hs) / 4 \\ \text{dimensionless; } \varepsilon \rightarrow 1/7 \text{ corresponds to Stokes waves at the breaking threshold}$$

Relative amplitude a:

$$a = As / h \\ \text{ratio of amplitude to depth}$$

Ursell number Ur :

$$Ur = (3 \cdot k \cdot H_s) / (2 \cdot k \cdot h)^3$$

governs the balance between dispersion and nonlinearity; $Ur \gg 1$ — nonlinear, $Ur \ll 1$ — linear

1.7.10 Spectral Parameters (Q , v , ϵ_w)

The function `calculate_spectrum_params` computes parameters from spectral moments. First a **Hann window** is applied to reduce edge effects:

$$S(\omega) = |\text{FFT}(\text{arr} \times w_{\text{Hann}})|$$

where $w_{\text{Hann}}[n] = 0.5 \cdot (1 - \cos(2\pi n/(N-1)))$

The spectrum is truncated at half-Nyquist: $\omega < \pi \cdot fs/2$. Spectral moments are computed:

$$m_n = \int \omega^n \cdot S(\omega) d\omega$$

$n = 0, 1, 2, 4$; numerical integration by the trapezoid rule

Goda peakedness parameter Q :

$$Q = \int \omega \cdot S^2(\omega) d\omega / m_0^2$$

$Q \rightarrow 1$ for a narrowband sinusoidal signal

Spectral narrowness parameter v :

$$v = \sqrt{(m_0 \cdot m_2 / m_1^2 - 1)}$$

$v = 0$ for a perfectly narrowband spectrum

Spectral bandwidth ϵ_w :

$$\epsilon_w = \sqrt{(1 - m_2^2 / (m_0 \cdot m_4))}$$

$\epsilon_w \rightarrow 0$ narrowband, $\epsilon_w \rightarrow 1$ broadband

1.7.11 Regularity Parameter p

The parameter p characterises how "regular" each individual wave is — whether it has a single maximum. For its computation an additional **half-Nyquist low-pass filter** is applied to suppress noise:

$$y_{\text{filtered}} = \text{IFFT}(\text{FFT}(\text{arr}), \text{zeroing } \omega > fs/4)$$

half-Nyquist = $fs/4$ Hz

Zero-crossings of the filtered signal define segments. Within each segment, **local maxima of $|y|$** are counted — points where $|y[c]| > |y[c-1]|$ and $|y[c]| > |y[c+1]|$. These are the extrema of each half-wave.

$$\rho = \min(N_segments / N_extrema, 1.0)$$

N_segments — number of half-waves; N_extrema — total extrema count across all half-waves

For a perfect sinusoid each segment has exactly one extremum, so $\rho = 1$. For irregular waves with multiple "humps", $\rho < 1$.

Spectral bandwidth derived from ρ :

$$\varepsilon_\rho = 2\sqrt{(1 - \rho) / (2 - \rho)}$$

alternative bandwidth measure; $\varepsilon_\rho = 0$ when $\rho = 1$

⚠ Note: The spectral bandwidth parameter ε should theoretically be independent of the measurement method, but in practice different estimators (ε_w and ε_ρ) may give somewhat different values for real sea states. Both are provided so they can be compared and the appropriate one chosen for a given analysis.

1.7.12 Benjamin–Feir Index (BFI)

The Benjamin–Feir Index characterises the susceptibility of the wave field to modulational instability and the formation of rogue ("freak") waves.

BFI_proper (full formula):

$$\text{BFI_proper} = \sqrt{(2\pi) \cdot \varepsilon \cdot Q \cdot \gamma}$$

ε — steepness, Q — Goda parameter, γ — nonlinear coefficient

BFI_goda (simplified Goda formula):

$$\text{BFI_goda} = \sqrt{(2\pi) \cdot \varepsilon \cdot Q}$$

without the nonlinear correction γ

Nonlinear coefficient γ (function of kh):

$$v = 1 + 2kh/\sinh(2kh)$$

$$a = -v^2 + 2 + 8(kh)^2 \cdot \cosh(2kh)/\sinh^2(2kh)$$

$$b = [\cosh(4kh) + 8 - 2\tanh^2(kh)] / [8\sinh^4(kh)] - [2\cosh^2(kh) + v/2]^2 / [\sinh^2(2kh) \cdot (kh/\tanh(kh) - v^2/4)]$$

$$\gamma = v \cdot \sqrt{(|b| / |a|)}$$

1.8 Program Output

Upon completion of Step 4 the program creates the following files in the Output folder:

File	Contents
<code>Parameters.csv</code>	All wave parameters for each 20-minute record: RMS, As, Hs, As_1/3, Hs_1/3, Tz, kh, k, ϵ , a, Ur, Q, v, ϵ_w , ρ , ϵ_ρ , γ , BFI_proper, BFI_goda
<code>Step4_Filtered.csv</code>	Surface displacement time series after removal of low-energy records and spike correction. Columns: timestamp, surface_displacement, reading_number
<code>Amplitudes_Heights.csv</code>	For each record: n_waves — wave count, amplitudes — individual amplitude array, heights — height array, semicolon-separated

All three files are stored in the `Output/` folder. From the "Pipeline Complete" window they can be exported in **.txt** (tab-separated) or **.mat** (MATLAB) format.

2. Step-by-Step User Guide

This chapter describes working with the program from launch to obtaining output files. Each section corresponds to one pipeline step. For each step the **required actions**, **on-screen display** and **saved output file** are described.

2.1 Preparation

2.1.1 System Requirements

The program runs on Windows 10/11.

- RAM: at least 4 GB (8 GB recommended for long recordings)
- Disk space: the Output folder is roughly the same size as the source .dat files

2.1.2 Launching the Program

Run `Data_Preprocessing_Pipeline.exe`

The main **Step 1** window opens. The progress indicator at the top of the window always shows the current pipeline step:

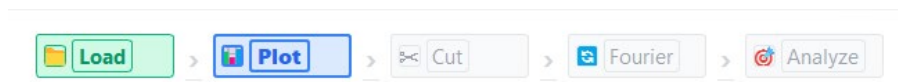


Fig. 3 — Progress indicator: active step highlighted in blue, completed steps in lighter blue

2.1.3 Output Folder and Checkpoint System

All intermediate and final files are saved in the `Output/` folder next to the program. On each launch the program **automatically checks** which files already exist and offers to resume from the corresponding step, skipping already completed ones.

This allows, for example, repeating only Step 4 with a different RMS threshold without recomputing the Fourier transform. Checkpoint files:

File	Unlocks step	Contents
<code>Step1_TXTtoCSV.csv</code>	Step 2	Merged time series with timestamps
<code>Step2_Zero_Mean.csv</code>	Step 3	Zero-mean trimmed series
<code>Step3_Transformed.csv</code>	Step 4	Series after Fourier filtering
<code>Parameters.csv</code>	—	Final output parameters

⚠ Note: "Clear Output" in the main menu deletes all intermediate files and allows starting over from scratch. The files `Parameters.csv`, `Step4_Filtered.csv` and `Amplitudes_Heights.csv` are not deleted — they are treated as the final result.

2.2 Step 1 — Loading and Converting Files

1 Load	2 Plot	3 Fourier	4 Analyze	✓ Done
--------	--------	-----------	-----------	--------

Goal: load .dat and .INFO files, merge them into a unified time series and save to `Step1_TXTtoCSV.csv`.

2.2.1 Adding Files

Files can be added in two ways:

1. **Drag and drop** .dat and .INFO files directly onto the dashed-border grey zone in the centre of the window.
2. **Click "Add Files"** and select files through the standard dialog.

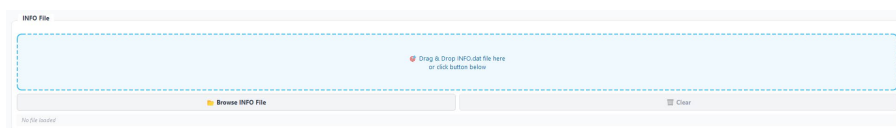


Fig. 4 — Step 1 window: drag & drop zone and file list

After adding, all loaded files appear in the list. Verify that there is **exactly one .INFO file** and one or more **.dat files**. If multiple .dat files are present they will be merged in alphabetical/chronological order of their names.

Tip: You can add .dat files one by one or select all at once in the dialog (Ctrl+A). Unwanted files can be removed from the list by selecting them and clicking "Remove selected".

2.2.2 Starting Conversion

Click **"Process & Convert"**. A progress bar appears. The program sequentially:

- Reads the .INFO file and extracts the sampling rate and start/end times
- Reads all .dat files and merges the values into one array
- Generates timestamps starting from the date and time in .INFO
- Splits the series into 20-minute blocks (reading_number = 1, 2, 3, ...)
- Saves the result to `Output/Step1_TXTtoCSV.csv`

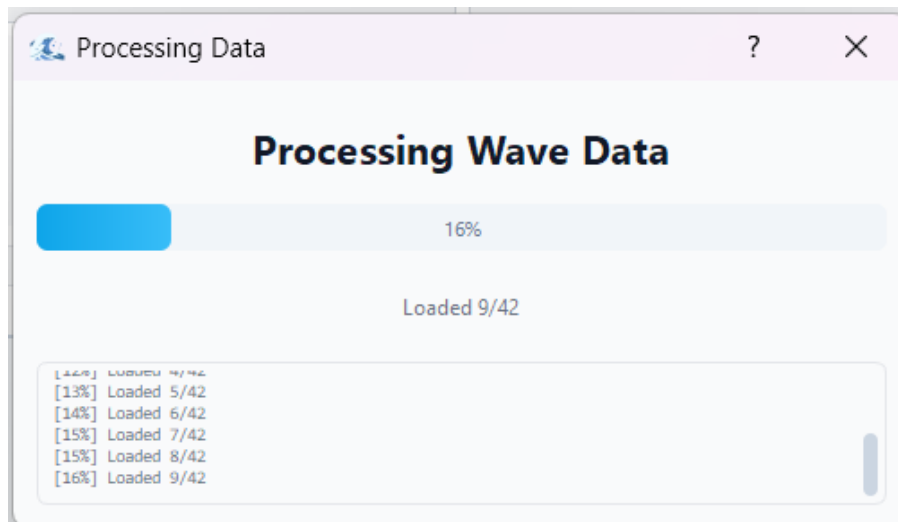


Fig. 5 — Progress bar during conversion

2.2.3 Completion Message

Upon completion a dialog appears with the results:

- **Total points** — total number of samples in the merged series
- **Total readings (20-min)** — number of complete 20-minute blocks
- **Sensor frequency** — sampling rate read from .INFO
- **Recording period** — start and end of the recording

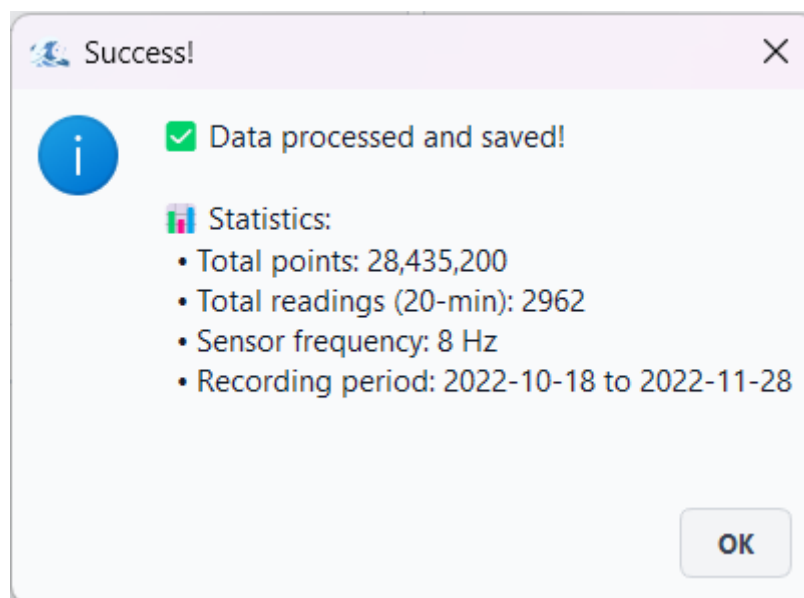


Fig. 6 — Completion dialog after successful conversion

After closing the dialog the program automatically moves to **Step 2**.

2.3 Step 2 — Visualisation and Trimming

1 Load	2 Plot	3 Fourier	4 Analyze	✓ Done
--------	--------	-----------	-----------	--------

Goal: visually inspect the recording, remove dive legs, and save the trimmed zero-mean signal to `Step2_Zero_Mean.csv`.

2.3.1 Reading the Graph

After data loading a surface displacement graph is displayed. By default a **subsamped version** ($\leq 10\,000$ points) is rendered for fast display. The X axis is time, the Y axis is the signal value in metres.



Fig. 7 — Step 2 main window: full recording with dive legs highlighted in red

Sections identified by the detector as dive or retrieval legs are highlighted in **red**. These are the parts of the recording when the sensor was still moving through the water column.

2.3.2 Manual Trimming

If the detector boundaries are correct — click **"Continue"**. To adjust them — click **"Manual trim"** (active only if legs were detected).

In manual trim mode:

1. Click **"Set start"** then click on the graph — a vertical blue line marks the selected point.
2. Click **"Set end"** then click on the graph — a second blue line appears.
3. Click **"Apply trim"** — data outside the selected range is removed.

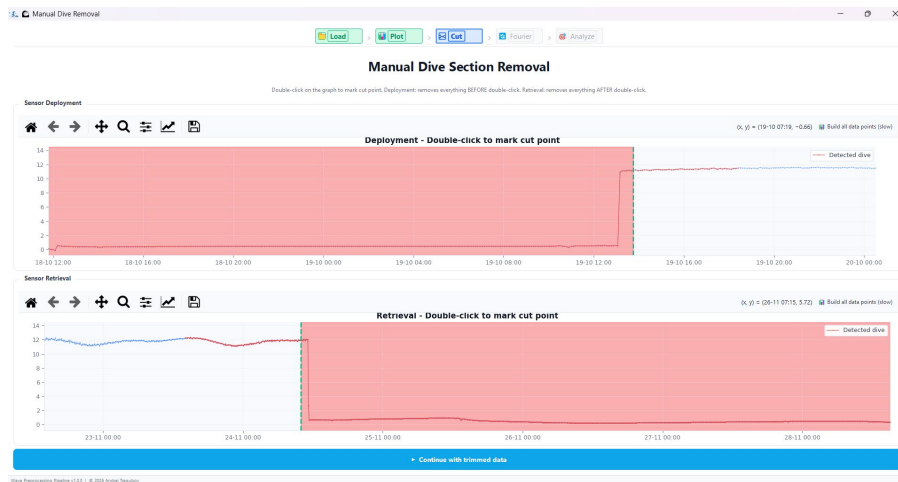


Fig. 8 — Manual trim mode: two vertical blue lines bounding the working section

Tip: Before clicking on the graph, press "Build all data points" in the toolbar — then your click will correspond to the true position in the full data, not the subsampled version.

2.3.3 What Happens to the Data

After confirming the trim, the program:

- Removes points outside the selected range
- Subtracts the mean value of each 20-minute record (zero-mean; the mean equals the sensor installation depth and is later used in the relative amplitude calculation)
- Saves the result to `Output/Step2_Zero_Mean.csv`
- Saves the subsampled cache to `Output/Step2_Visualization.csv`

The program then automatically moves to **Step 3**.

2.4 Step 3 — Fourier Filtering

1 Load	2 Plot	3 Fourier	4 Analyze	✓ Done
--------	--------	-----------	-----------	--------

Goal: remove long-period components (tides, drift) from the signal and retain only wind waves in `Step3_Transformed.csv`.

2.4.1 Spectrum Computation

On entering Step 3 the program automatically computes the full Fourier spectrum of the entire signal. This may take several seconds for long recordings. The **spectrum window with two panels** then opens:



Fig. 9 — Step 3 window: two spectrum panels and Apply button

- **Upper panel** — angular frequency range from 0.1 rad/s upward, linear scale. Wind wave peaks are visible here.
- **Lower panel** — range 0–0.1 rad/s on a logarithmic vertical scale. Tidal and infragravity components are visible here.

Additionally, a **spectrogram** (short-time Fourier transform) can be built in the upper part of the window. The user selects the window length — longer windows give better frequency resolution, shorter windows give better time resolution.

2.4.2 Cutoff Frequency Selection

Double-click on the **lower panel** to select the cutoff frequency ω_c . After the click:

- A vertical red line appears at the selected frequency on both panels
- The corresponding period is displayed: $T = 2\pi / \omega_c$ (in seconds and minutes)
- The area to the left of the line is shaded — these components will be removed from both the real and imaginary (mirrored) parts of the spectrum
- The **"Apply & Continue"** button activates

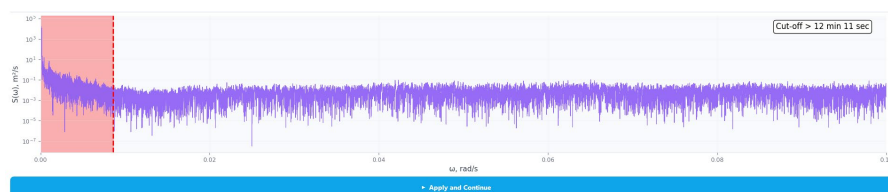


Fig. 10 — Lower spectrum with selected cutoff: red vertical line and shaded zone

⚠ Note: Recommended selection: cut at a period of 10 minutes ($\omega \approx 0.0105$ rad/s). This removes tidal and infragravity components while leaving all wind waves intact. If in doubt — cut further right rather than left: it is better to leave a small residual trend than to clip the longest wind waves.

2.4.3 Applying the Filter and Comparison Window

Click **"Apply & Continue"**. The program:

1. Appends mirror tails to the signal to suppress edge artefacts
2. Applies FFT to the extended signal
3. Zeroes components below the cutoff
4. Applies inverse FFT
5. Discards the tails — returns the signal to its original length
6. Saves the result to `Step3_Transformed.csv`

The **"Before/After"** comparison window then opens automatically. Both signals are overlaid: **orange** — original before filtering, **blue** — filtered. Verify that:

- The long-period trend (slow rise/fall) has disappeared
- The wave shape is preserved — the blue signal "follows" the orange one, just without the trend
- There are no strong oscillations at the record edges (if there are — try a different cutoff frequency)

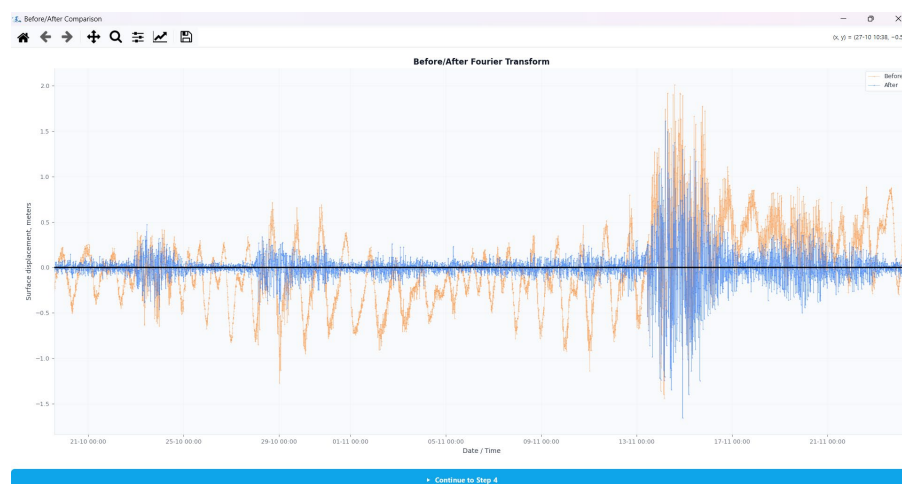


Fig. 11 — Before/After window: orange original and blue filtered signal

After closing the comparison window the program moves to **Step 4**.

2.5 Step 4 — Analysis and Parameter Computation

1 Load	2 Plot	3 Fourier	4 Analyze	✓ Done
--------	--------	-----------	-----------	--------

Goal: pass through all 20-minute records, apply optional cleaning, and compute the full set of wave parameters.

2.5.1 Settings Before Starting

Before clicking Start Processing, three options can be configured:

Option	Parameter	Description
Remove recordings with RMS <	0.015 m	Excludes 20-minute records where wave energy is below the threshold — quiet periods. The RMS threshold is set manually in the adjacent field. Recommended: enable.
Remove spikes	—	Detects and corrects individual anomalous points (jumps $> 6\sigma$) by linear interpolation between neighbours. Recommended: enable.
Spline interpolation	target Hz	Resamples the signal to a higher frequency using a cubic spline before computing amplitudes. Useful for low-rate sensors (1 Hz) — improves accuracy of the per-half-wave maximum estimate.

2.5.2 Real-Time Processing

Click "► **Start Processing**". A progress bar opens, then a **real-time graph** begins building. Each 20-minute record is processed sequentially and displayed as it is computed:

- **Green sections** — records that passed the RMS filter and are included in the parameter computation
- **Red sections (semi-transparent)** — records excluded as "quiet" (RMS below threshold)
- **Orange circles** — corrected spikes (if spike removal is enabled)

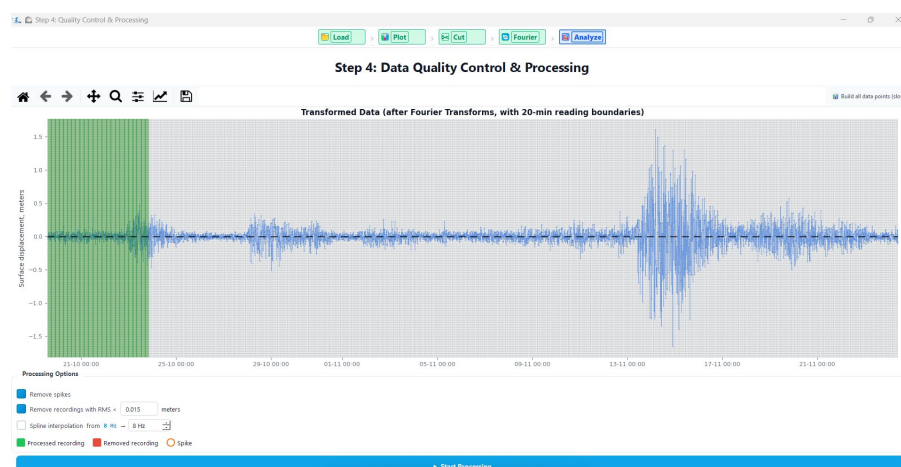


Fig. 12 — Step 4 real-time graph: green and red sections, spike circles

After the last record is processed, the **"Pipeline Complete"** window opens automatically.

2.5.3 Pipeline Complete Window

The window shows the final processing statistics:

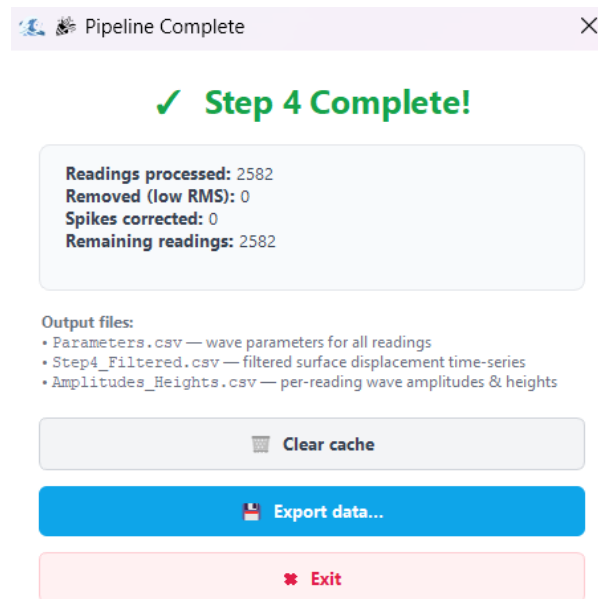


Fig. 13 — Pipeline Complete window: statistics and output file list

The **"Export"** button in this window saves the results in the desired format.

2.6 Exporting Results

The **"Export"** button in the Pipeline Complete window launches a format selection dialog. Two formats are available:

- **.txt (tab-separated)** — plain text file with tab delimiters. Opens in Excel, MATLAB, Python, or any text editor.
- **.mat (MATLAB)** — binary MATLAB v5 format file. Contains the same data as matrices and cell arrays. Loaded directly with `load('filename.mat')` in MATLAB.

After selecting the format a destination folder dialog opens. Three files are exported:

File	Contents
<code>Parameters</code>	Table: one row = one 20-minute record. Columns — all computed parameters (RMS, Hs, Tz, kh, BFI, etc.).
<code>Step4_Filtered</code>	Surface displacement time series after removal of quiet records and spike correction. Columns: timestamp, surface_displacement, reading_number.
<code>Amplitudes_Heights</code>	For each record: <code>n_waves</code> — wave count, amplitudes and heights arrays. In .mat format stored as variable-length cell arrays.

2.6.1 Loading Results in MATLAB

All three .mat files are loaded with the standard `load` command.

Parameters.mat — wave parameter table. Each variable is a column vector of length N (number of 20-minute records):

```
data = load('Parameters.mat');
reading = data.reading_number; % record numbers, Nx1 vector
Hs      = data.Hs;             % significant wave height, m
Hs13    = data.Hs_1_3;        % H 1/3 from individual waves, m
Tz      = data.Tz;             % zero-crossing period, s
BFI     = data.BFI_proper;     % Benjamin-Feir Index
rho     = data.rho;            % regularity parameter
```

Step4_Filtered.mat — cleaned time series. Contains three variables:

```
ts = load('Step4_Filtered.mat');
eta  = ts.surface_displacement; % surface displacement, m
reading = ts.reading_number;    % 20-min record number
% Time is stored as days from 0000-01-01 00:00:00
% To convert to standard MATLAB datenum (from 0000-01-00):
t_dn  = ts.timestamp + 1;       % +1 day correction
t_dt  = datetime(t_dn, 'ConvertFrom', 'datenum');
% Example: plot the time series for record 1
mask = reading == 1;
plot(t_dt(mask), eta(mask));
```

Amplitudes_Heights.mat — individual amplitudes and heights. Variables `amplitudes` and `heights` are stored as **cell arrays** — each cell contains a variable-length vector for one record:

```
ah = load('Amplitudes_Heights.mat');
n_waves = ah.n_waves;           % wave count per record
% Amplitudes and heights for record 1:
A1 = ah.amplitudes{1};          % amplitude vector, m
H1 = ah.heights{1};             % wave height vector, m
% Height histogram across the full dataset:
all_H = vertcat(ah.heights{:});
histogram(all_H, 30);
```

⚠ Note: The timestamp variable in `Step4_Filtered.mat` is stored as days from 0000-01-01. Standard MATLAB datenum counts from 0000-01-00 (one day earlier), so a +1 correction is needed before converting via `datetime(..., 'ConvertFrom', 'datenum')`.

2.6.2 Loading Results in Python

Parameters.txt:

```
import pandas as pd
params = pd.read_csv('Parameters.txt', sep='\t')
Hs = params["Hs"].values
Tz = params["Tz"].values
```

Step4_Filtered.txt:

```
ts = pd.read_csv('Step4_Filtered.txt', sep='\t')
ts['timestamp'] = pd.to_datetime(ts['timestamp'])
rec1 = ts[ts['reading_number'] == 1]
import matplotlib.pyplot as plt
plt.plot(rec1['timestamp'], rec1['surface_displacement'])
```

Amplitudes_Heights.mat:

```
import scipy.io as sio, numpy as np
ah = sio.loadmat('Amplitudes_Heights.mat')
H1 = ah["heights"][0, 0].flatten() # heights for record 1
all_H = np.concatenate([ah["heights"][0,i].flatten()
                        for i in range(ah["heights"].shape[1])])
```

— End of Manual —